



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-48569 Visual Studio Code Security Feature Bypass Vulnerability

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-10
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-48569 Visual Studio Code Security Feature Bypass Vulnerability - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-10 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

CVE-2026-48569 is a high-severity Visual Studio Code security feature bypass vulnerability published by Microsoft on 2026-06-09 and updated on 2026-06-10 when Microsoft revised the security-updates build number. Microsoft, the CVE Program CNA record, and NVD align that the flaw is an improper input validation issue with a relative-path-traversal dimension (CWE-20, CWE-23) that allows an unauthorized attacker to bypass a security feature locally. The reviewed public record assigns CVSS v3.1 base score 7.1 / HIGH, requires user interaction, and does not indicate availability-heavy impact; the primary risk is unauthorized modification of sensitive local content or security-relevant developer configuration.

The strongest public clue about the vulnerable implementation is the only publicly visible VS Code issue shipped in the 1.123.2 recovery milestone: GitHub issue #320660, which extends a file-write redirection guard so auto-approved writes are no longer allowed when destination paths use POSIX ~ expansion or Windows %ENVVAR% expansion. That issue strongly suggests the CVE affected trust boundaries around agent-assisted or otherwise auto-approved file-write workflows. Microsoft has not explicitly stated in the advisory that issue #320660 is the root cause, so this mapping should be treated as a high-value but still inferential technical correlation rather than a vendor-confirmed exploit narrative.

For defenders, the practical risk is concentrated on developer endpoints where users rely on VS Code guardrails to prevent writes to sensitive local paths. A malicious repository, prompt-injection payload, extension, or locally present attacker could potentially steer a user into a file-write action that lands in shell profiles, SSH configuration, startup locations, or other trust-sensitive paths if the editor incorrectly treats those redirected destinations as safe for auto-approval. Microsoft states the issue was not publicly disclosed and not exploited at release, and the reviewed CISA KEV feed did not list the CVE as of 2026-06-11. Immediate priority should therefore be upgrading to 1.123.2 or later, reviewing AI-assisted write workflows, and hunting for suspicious VS Code-driven writes to persistence- or credential-relevant paths.

Executive takeaway	Assessment
Business impact	The flaw can undermine a VS Code safety control and enable writes to security-sensitive local files, creating risk of developer-environment persistence, credential exposure, or follow-on code execution when modified profiles or startup paths are later invoked.
Attacker requirement	Local attack path with no privileges required, but user interaction is required (AV:L/PR:N/UI:R).
Public exploitation status	No reviewed authoritative source confirmed in-the-wild exploitation as of 2026-06-11.
Immediate priority	Upgrade VS Code to 1.123.2+, reduce auto-approval exposure in agent workflows, and review recent writes by VS Code processes to shell, SSH, and startup-related paths.

2. Intelligence Requirement

This review addresses the following intelligence requirement: what does the current public record say about CVE-2026-48569, which Visual Studio Code deployments and workflows are exposed, how credible is near-term exploitation risk, and what should defenders do immediately to reduce abuse opportunities on developer endpoints?

Question	Assessment
What is the vulnerability?	A Visual Studio Code security feature bypass caused by improper input validation with a relative-path-traversal component, allowing a local unauthorized attacker to bypass a safety control.
Which versions are affected?	The CNA record marks Visual Studio Code versions 1.0.0 through < 1.123.2 as affected.

What access does the attacker need?	Local attack path with no privileges required, but the public CVSS vector requires user interaction.
What is the practical impact?	The most plausible public outcome is unauthorized writing to sensitive local paths that VS Code should have treated as high-risk, which can enable persistence, configuration tampering, or credential-path manipulation.
Is exploitation publicly confirmed?	No. Microsoft states the issue was not publicly disclosed and not exploited at release, and the reviewed KEV snapshot did not list it.
What matters most to defenders?	Patch quickly, reduce unsafe auto-approval patterns in AI-assisted development workflows, and investigate whether VS Code wrote to shell profiles, SSH paths, or startup locations on unpatched systems.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Update Guide entry	Canonical advisory title, current public status page, and remediation linkage.	High
Microsoft CVRF OData update mapping	Ties the CVE to the June 2026 security release and confirms the advisory chronology.	High
Microsoft June 2026 CVRF document	Authoritative CVSS vector, exploitability assessment, affected product identity, fixed build `1.123.2`, and revision history.	High
CVE Program CNA API record	Confirms affected-version range, weakness mapping, CVSS vector, and publication timing.	High
NVD JSON 2.0 record	Corroborates normalized vulnerability metadata and current analysis state.	High
Visual Studio Code 1.123 release notes	Confirms `1.123.2` as the public maintenance build users were directed to install.	Medium
Public VS Code GitHub issue `#320660` in milestone `1.123.2`	Provides the strongest public implementation clue: path-expansion bypass of a file-write redirection guard.	Medium
CISA KEV JSON feed snapshot	Confirms the CVE was not listed in KEV at review time.	High

Confidence is high on the existence of the vulnerability, the affected-version range, the fixed build, and the absence of reviewed public exploitation evidence because those facts are directly supported by Microsoft, the CNA record, NVD, and CISA. Confidence is only medium on the exact underlying bug mechanics because Microsoft's public advisory text remains terse. The GitHub issue and release-milestone evidence strongly suggest that the vulnerable control involved auto-approved file writes to `~`-expanded or `%ENVVAR%`-expanded destinations, but Microsoft does not explicitly name that issue in the CVE record.

That distinction matters operationally. The reviewed public sources support immediate defensive action and a realistic threat model, but they do not support overconfident claims about a single exploit chain, proof-of-concept tooling, or a named adversary campaign. Detection and hunting guidance in this report is therefore behavior-based and focused on likely misuse of sensitive local file paths rather than a brittle exploit signature.

4.1 Core Vulnerability Metadata

Field	Value
CVE	`CVE-2026-48569`
Vulnerability name	`Visual Studio Code Security Feature Bypass Vulnerability`
Weakness	`CWE-20` Improper Input Validation; `CWE-23` Relative Path Traversal
Severity	`7.1 / HIGH` (CVSS v3.1)
CVSS v3.1 vector	`AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N`
Microsoft temporal vector	`CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N/E:U/RL:O/RC:C`
Attack vector	Local
Privileges required	None
User interaction	Required
Scope	Changed
Impact profile	High confidentiality impact, low integrity impact, no availability impact in the reviewed public record
Microsoft severity label	Important
Product	Visual Studio Code
Affected versions	`>= 1.0.0` and `< 1.123.2`
Fixed version	`1.123.2`
Public disclosure at release	No
Exploitation at release	No
Microsoft exploitability assessment	Exploitation Less Likely
Customer action required	Yes
Acknowledged finder	Anonymous
Intake event reviewed	2026-06-10 build-number update

4.2 Public Implementation Clue and Exposure Pattern

Evidence point	What it suggests	Confidence
Microsoft advisory language cites improper input validation and relative path traversal.	The vulnerable control likely mishandled destination-path normalization or trust	High

	decisions rather than a memory-safety flaw.	
Public VS Code issue `#320660` says auto-approval should be blocked when destination paths contain `~` or `%...%` expansion syntax.	The most plausible public implementation detail is a bypass in a file-write redirection guard protecting sensitive absolute-path writes.	Medium
The issue adds regression tests for bash and PowerShell path-expansion forms.	The exposure is cross-platform and relevant to both POSIX-style home-directory paths and Windows environment-variable expansion.	Medium
The fixed public build is `1.123.2`, a focused maintenance release.	Organizations cannot assume the base `1.123.0` or `1.123.1` builds are sufficient; explicit version verification matters.	High

4.3 Likely High-Risk Destinations if the Bypass Is Abused

Destination pattern	Why it matters
`~/bashrc`, `~/zshrc`, `~/profile`, `~/config/fish/config.fish`	Modifying shell startup files can create persistence or run attacker-controlled commands in future developer sessions.
`~/ssh/config`, `~/ssh/authorized_keys`	Changes here can redirect SSH behavior, weaken trust decisions, or add durable remote-access material.
PowerShell profile paths such as `%USERPROFILE%\Documents\PowerShell\Microsoft.PowerShell_profile.ps1`	Startup script modification can provide persistence and trusted-script execution on Windows developer endpoints.
`%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup`	Startup-folder writes can create user-level persistence on Windows.
VS Code user settings / task / workspace configuration files	A malicious write may weaken future trust prompts, alter tooling behavior, or stage additional execution paths.

4.4 Analyst Findings

Finding	Defensive implication
The CVE is local and requires user interaction, which lowers mass remote-exploitation risk.	Patch urgency should be high on developer systems, but public messaging should avoid overstating internet-scale exploitation.
The affected-version range is broad (`< 1.123.2`) and spans all long-lived user installations rather than a niche feature branch.	Fleet inventory and upgrade verification matter more than relying on a small exposure assumption.
The best public technical clue points to agent-assisted or auto-approved write workflows.	Teams using AI coding assistants, autonomous edit modes, or low-friction approval settings should be treated as the most operationally relevant population.
The likely abuse path targets trust-sensitive local files rather than causing immediate availability loss.	Hunting should focus on file modifications to startup, shell, SSH, and profile paths rather than crash or denial-of-service indicators.
No reviewed source published stable exploit IOCs, malware samples, or named threat-actor attribution.	Detection must remain behavior-based, and incident triage should correlate file lineage, user workflow context, and VS Code version state.

5. Threat Context

CVE-2026-48569 is best modeled as a developer-workstation trust-boundary failure rather than a classic remote initial-access vulnerability. The public record indicates that the attacker does not need privileges, but it also requires user interaction and a local attack context. In practice, that combination points toward scenarios where a user is induced to let Visual Studio Code perform or approve an action that appears safe because a built-in safety control should have blocked the destination path. If the technical correlation to issue `#320660` is correct, the failure mode is especially relevant to AI-assisted workflows that generate or apply file writes on a user's behalf.

The most concerning environments are those where VS Code is used on privileged developer endpoints, jump boxes, build systems with user-interactive sessions, or administrator workstations that already hold cloud credentials, SSH material, or source-code signing context. In those environments, a write into a shell profile, PowerShell profile, startup folder, or SSH configuration file can create durable follow-on access even if the original exploit condition itself was only a local safety-control bypass.

No reviewed authoritative source tied this CVE to a named threat actor, malware family, proof-of-concept release, or active campaign as of 2026-06-11. The reviewed CISA KEV snapshot did not include the CVE. That absence should be treated as time-bounded negative evidence for current widespread abuse, not as proof that exploitation is implausible. Security-feature-bypass flaws in developer tooling can become disproportionately valuable because they target systems that already hold secrets, code, and privileged workflow access.

ATT&CK mapping	Relevance	Confidence
`T1546.004` Unix Shell Configuration Modification	Plausible downstream persistence technique if the bypass is used to write to `~/.bashrc`, `~/.zshrc`, or similar startup files. This is scenario-dependent and not publicly confirmed as observed exploitation.	Medium
`T1098.004` SSH Authorized Keys	Plausible if an attacker abuses the bypass to add or alter `authorized_keys` material on developer endpoints. This is a potential consequence, not an observed technique from the reviewed public record.	Low
`T1547.001` Registry Run Keys / Startup Folder	Plausible Windows persistence outcome if expanded-path writes land in a Startup folder or equivalent logon-triggered location.	Low
Additional ATT&CK techniques	Not assigned with high confidence because the public source set does not document a complete intrusion chain or in-the-wild tradecraft.	High

6. Detection and Hunting

Because public reporting does not currently expose exploit payloads, packet signatures, or stable IOCs, the most defensible monitoring approach is layered: identify unpatched VS Code installations, detect VS Code-driven writes to trust-sensitive locations, and review whether AI-assisted or agent-assisted file actions preceded unexpected profile, SSH, or startup changes.

6.1 Detection Logic Summary

Signal	Telemetry source	Analytic goal
--------	------------------	---------------

Endpoints running Visual Studio Code `< 1.123.2`	Software inventory, EDR device inventory, package-management telemetry	Identify exposed systems and prioritize upgrades
VS Code processes writing to shell profiles, SSH paths, PowerShell profiles, or Startup folders	EDR file telemetry, Sysmon file-create events, macOS/Linux audit telemetry, FIM	Detect likely abuse of redirected sensitive-path writes
New or modified files under `~/ssh/`, shell-profile paths, or user startup directories shortly after a VS Code session	File integrity monitoring, timeline correlation, EDR process/file lineage	Surface persistence or configuration-tampering outcomes
VS Code or helper processes spawning shells or script interpreters near suspicious file writes	EDR process telemetry, Sysmon Event ID 1, Windows 4688	Add context for user-assisted or tool-assisted file-write abuse
Approval, prompt, or extension activity logs containing `~` or `%ENVVAR%` destination patterns where available	Local editor logs, extension logs, SIEM ingestion of developer-tool telemetry	Support root-cause review when suspicious file changes are found

Analyst caveat: the reviewed public evidence supports high-value hunting pivots, not exploit-confirmation logic. Many developer endpoints legitimately use VS Code to edit shell profiles or dotfiles. Triage should focus on unusual parent/child process lineage, suspicious timing, surprising path choice, and whether the affected system remained below `1.123.2` at the time of the write.

6.2 Example Sigma Logic

```

title: Suspicious Sensitive-Path Write by Visual Studio Code Process
id: 48569b42-vscode-sensitive-path-write
status: experimental
description: Detects VS Code writing to sensitive shell, SSH, or startup paths that may merit review during CVE-2026-48569 response activity.
logsource:
  category: file_event
  product: windows
detection:
  selection_process:
    Image|endswith:
      - '\\Code.exe'
      - '\\code.exe'
      - '\\Code - Insiders.exe'
      - '\\code-insiders.exe'
  selection_target:
    TargetFilename|contains:
      - '\\.ssh\\'
      - '\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\'
      - '\\Documents\\PowerShell\\'
      - '\\WindowsPowerShell\\'
      - '\\.vscode\\'
  condition: selection_process and selection_target
falsepositives:
  - expected administrator or developer customization of profiles and SSH configuration
  - legitimate VS Code extension or settings changes reviewed and approved by the user
level: medium

```

6.3 Example Hunt Queries

Microsoft Defender XDR / Sentinel-style hunt for suspicious VS Code writes to sensitive paths:

```

DeviceFileEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("Code.exe", "code.exe", "Code - Insiders.exe", "code-insiders.exe", "code", "code-insiders")
| where FolderPath has_any (

```



```

"\\.ssh\\",
"\\Documents\\PowerShell\\",
"\\Documents\\WindowsPowerShell\\",
"\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\",
"/.ssh/",
"/.bashrc",
"/.zshrc",
"/.profile",
"/.config/fish/"
)
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, FolderPath,
FileName, SHA1, AccountName
| order by Timestamp desc

```

Timeline-correlation hunt for VS Code processes followed by shell or script execution from recently modified profile locations:

```

let sensitiveWrites = DeviceFileEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("Code.exe", "code.exe", "Code - Insiders.exe", "code-
insiders.exe", "code", "code-insiders")
| where FolderPath has_any ("\\.ssh\\", "/.ssh/", "/.bashrc", "/.zshrc", "/.profile", "\\Documents\\
PowerShell\\", "\\Startup\\")
| project WriteTime=Timestamp, DeviceId, DeviceName, AccountName, FolderPath, FileName, SHA1;
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("powershell.exe", "pwsh.exe", "bash", "zsh", "sh", "cmd.exe", "ssh.exe")
| join kind=inner sensitiveWrites on DeviceId
| where Timestamp between (WriteTime .. WriteTime + 2h)
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, FolderPath,
FileName1=FileName1, WriteTime
| order by Timestamp desc

```

Splunk example for suspicious VS Code-sensitive-path file activity:

```

index=edr OR index=sysmon
(Image="*\\Code.exe" OR Image="*\\code.exe" OR Image="*\\code-insiders.exe" OR process_name="code")
(TargetFilename="*\\.ssh\\*" OR TargetFilename="*\\Documents\\PowerShell\\*" OR TargetFilename="*\\
Startup\\*" OR TargetFilename="*/.ssh/*" OR TargetFilename="*/.bashrc" OR TargetFilename="*/.zshrc" OR
TargetFilename="*/.profile")
| stats count min(_time) as firstTime max(_time) as lastTime values(Image) as image values(CommandLine)
as commandLine values(TargetFilename) as target by host user
| convert ctime(firstTime) ctime(lastTime)
| sort - lastTime

```

6.4 Hunting Guidance

Hunt question	Why it matters
Which endpoints were still running VS Code below `1.123.2` after 2026-06-09?	These systems form the highest-priority review and upgrade set.
Did VS Code write to shell profiles, SSH paths, or startup folders on those endpoints?	That is the most plausible public abuse pattern for this CVE.
Were sensitive-path writes followed by shell launches, SSH activity, or unusual developer-authentication events?	Those sequences can distinguish routine customization from persistence or credential-path abuse.
Did AI-assisted editing, auto-approval, or agent sessions occur shortly before the write where local logs preserve that context?	This helps confirm whether low-friction file-approval workflows were part of the chain.
Were suspicious writes dismissed as normal dotfile editing without checking version state and path intent?	Legitimate development activity can mask malicious modifications if investigators do not correlate the file destination and initiating process.

7. Recommendations

Priority	Owner	Action
Critical	Endpoint Engineering / Developer Platform	Upgrade Visual Studio Code to `1.123.2` or later across all managed developer systems, including insider or manually installed variants.
Critical	Vulnerability Management	Verify remediation by installed VS Code version, not by assumption that June 2026 patching or extension updates covered the issue.
High	Engineering Enablement / Security Champions	Temporarily reduce or disable unsafe auto-approval patterns in agent-assisted or autonomous editing workflows until patched versions are confirmed.
High	SOC / Detection Engineering	Deploy focused monitoring for VS Code writes to shell profiles, SSH directories, PowerShell profiles, and Startup-folder locations on endpoints that were unpatched after 2026-06-09.
High	Incident Response	Retrospectively review sensitive-path file modifications by VS Code processes since 2026-06-09, especially on privileged developer workstations, CI jump hosts with user sessions, and administrator endpoints.
Medium	IAM / DevSecOps	Review where developer endpoints store SSH keys, cloud credentials, and code-signing context, and prioritize those systems for patch verification and hunting.
Medium	Engineering Leadership	Reinforce guidance that users should scrutinize tool-generated file destinations, especially when prompts or repositories attempt to write outside the project tree.
Medium	Threat Intelligence	Re-check Microsoft advisories, VS Code release notes, and KEV status in subsequent patch cycles for exploit-status changes or stronger technical disclosure.

Additional analyst guidance:

- Treat the absence of public exploitation evidence as a calibration input, not as a reason to deprioritize developer-endpoint patching where sensitive credentials or privileged workflows exist.
- If immediate upgrade is not possible, document the exception, disable high-risk approval shortcuts where feasible, and add temporary monitoring around sensitive-path writes by VS Code processes.
- Do not overfit detections to a single file path. The security significance of this CVE comes from the trust boundary being bypassed, so defenders should monitor categories of sensitive destinations rather than one hard-coded artifact.

8. References

1. Microsoft Security Update Guide, CVE-2026-48569: <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-48569>
2. Microsoft CVRF OData update mapping for CVE-2026-48569:
<https://api.msrc.microsoft.com/cvrf/v3.0/updates/CVE-2026-48569>
3. Microsoft June 2026 CVRF document: <https://api.msrc.microsoft.com/cvrf/v3.0/cvrf/2026-Jun>
4. CVE Program CNA API record: <https://cveawg.mitre.org/api/cve/CVE-2026-48569>
5. NVD JSON 2.0 API query: <https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-48569>
6. Visual Studio Code June 2026 release notes (`1.123` / update `1.123.2`): https://code.visualstudio.com/updates/v1_123
7. Public VS Code issue `#320660`, `Block auto-approval of file-write redirection to ~ and % paths`:
<https://github.com/microsoft/vscode/issues/320660>
8. CISA Known Exploited Vulnerabilities JSON feed:
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json